

Data Structures & Systems Design (SE Intern) - Detailed Explanation

Problem 1: LRU Cache

What is the question asking?

We need to build a cache that has limited capacity. When the cache becomes full and a new item is added, the cache should automatically remove the item that has not been used for the longest time. This item is called the Least Recently Used (LRU) item.

The cache must support two operations:

1. `get(key)` - Return the value if the key exists, otherwise return -1.
2. `put(key, value)` - Insert or update a value.

The important requirement is that both operations must work in $O(1)$ time.

How did I solve it?

I used two data structures together:

- Hash Map (Dictionary)
- Doubly Linked List

Why a Hash Map?

A hash map allows us to find any key instantly. Instead of searching through all elements, we can directly access the required node in $O(1)$ time.

Why a Doubly Linked List?

The linked list keeps track of usage order. The most recently used item is kept at the front and the least recently used item stays at the end. A doubly linked list allows insertion, deletion, and movement of nodes in $O(1)$ time.

Working Example:

Capacity = 3

Add A, B, C

Order: $C \rightarrow B \rightarrow A$

If A is accessed using `get(A)`:

Order becomes: $A \rightarrow C \rightarrow B$

Now B is the least recently used item.

If D is added and cache is full:

Remove B and insert D.

Order becomes: $D \rightarrow A \rightarrow C$.

Why does this achieve $O(1)$?

The hash map gives direct access to a node. The linked list lets us move or remove a node without searching. Therefore `get()` and `put()` both run in $O(1)$ time.

Problem 2: Event Scheduler

What is the question asking?

We are given a list of events such as:

(9,10), (10,11), (11,12)

We must answer two questions:

1. Can one person attend all events?
2. What is the minimum number of rooms required?

Part 1: can_attend_all(events)

How did I solve it?

First, I sort all events by start time. Then I compare each event with the previous event.

If Current Start Time < Previous End Time, the meetings overlap and one person cannot attend both meetings.

Example:

(9,11) and (10,12)

$10 < 11$, therefore overlap exists.

Answer = False.

If there is no overlap between any meetings, answer = True.

Part 2: min_rooms_required(events)

How did I solve it?

I used a Min Heap (Priority Queue).

The heap stores the ending times of meetings currently occupying rooms. The smallest ending time always remains at the top of the heap.

For every new meeting:

- If the earliest room becomes free before the meeting starts, reuse that room.
- Otherwise allocate a new room.

Example:

Meetings:

(9,12), (10,11), (11,13)

Meeting 1 uses Room 1.

Heap = [12]

Meeting 2 starts at 10 while Room 1 is busy until 12.

Need Room 2.

Heap = [11,12]

Meeting 3 starts at 11.

The earliest ending room ends at 11.

That room becomes free and can be reused.

Therefore only 2 rooms are needed.

Complexity Analysis

LRU Cache:

get() = $O(1)$

put() = $O(1)$

Space = $O(\text{capacity})$

Event Scheduler:

Sorting = $O(n \log n)$

Heap Operations = $O(\log n)$
Overall Time = $O(n \log n)$
Space = $O(n)$

Trade-Offs

A hash map alone provides fast lookup but cannot maintain usage order. A linked list maintains order but searching is slow. Combining both structures provides fast lookup and efficient ordering, which is why this design is commonly used in LRU caches.

Future Improvement

To assign room names such as Room A, Room B, or Room C, store both the room number and ending time in the heap. When a room becomes available, reuse its room ID for future meetings.

Concurrency

If multiple threads access the cache at the same time, they may modify the linked list and hash map simultaneously. To make the cache thread-safe, use a lock (mutex) around `get()` and `put()` operations so only one thread modifies the cache at a time.